



**CHANDIGARH
UNIVERSITY**

Discover. Learn. Empower.

**UNIVERSITY INSTITUTE OF
COMPUTING**

**CASE STUDY
ON
YOUTUBE TRENDING ANALYSIS
PYTHON PROGRAMMING
25CAH-221**

SUBMITTED BY:

Name : S S SAHIL

UID : 25BCD10109

Branch : BCA (DS)

Section: 25BCD2(B)

Semester : 3

SUBMITTED TO:

Name : MISS. PREETI

D.O.SUBMISSION : 08/10/2026

Designation: PROFESSOR

Sign:_____

Case Study: YouTube Video Data Analysis and Visualization

1. Problem Statement

YouTube is one of the largest video-sharing platforms, where millions of videos are uploaded in different categories such as Music, Gaming, Education, Technology, and Comedy. The performance of a YouTube video can be measured using different factors such as the number of views, likes, and comments.

Analyzing this information can help us understand how videos perform and identify patterns between different categories. For example, it can help determine which category receives more views and whether videos with more views also receive more likes and comments.

The main objective of this case study is to analyze a small YouTube dataset using **NumPy and Pandas** and represent the results using **Matplotlib and Seaborn**. Different statistical calculations and visualization techniques are used to understand the data clearly.

2. Dataset Description

A dataset is a collection of related information that can be used for analysis. In this case study, a small dataset containing information about **10 YouTube videos** has been created.

The dataset contains information about the category of each video, its views, likes, and comments. These variables help us understand the popularity and engagement of the videos.

Variables in the Dataset

Variable	Description	Data Type
Category	Category of the YouTube video	String
Views	Number of times the video was viewed	Integer
Likes	Number of likes received	Integer
Comments	Number of comments received	Integer

The dataset contains five categories:

- Music
- Gaming
- Education
- Tech
- Comedy

There are two videos from each category, giving a total of **10 records**.

Dataset

Category	Views	Likes	Comments
----------	-------	-------	----------

Music	50,000	5,000	500
-------	--------	-------	-----

Gaming	80,000	8,000	800
--------	--------	-------	-----

Education	120,000	12,000	1,200
-----------	---------	--------	-------

Tech	200,000	20,000	2,000
------	---------	--------	-------

Comedy	150,000	15,000	1,500
--------	---------	--------	-------

Music	90,000	9,000	900
-------	--------	-------	-----

Gaming	180,000	18,000	1,800
--------	---------	--------	-------

Education	100,000	10,000	1,000
-----------	---------	--------	-------

Tech	250,000	25,000	2,500
------	---------	--------	-------

Comedy	140,000	14,000	1,400
--------	---------	--------	-------

3. Data Cleaning Using Pandas

Data cleaning is an important step in data analysis. Raw data may contain missing values, duplicate records, incorrect data types, or inconsistent values. Cleaning the data before analysis helps improve the accuracy of the results.

Pandas provides several functions that make it easy to identify and handle these problems.

3.1 Missing Values

Missing values are values that are not available in a dataset. They can occur because information was not entered or was unavailable.

Pandas can be used to check for missing values:

```
print(df.isnull().sum())
```

The result showed that there were **no missing values** in the final dataset. Therefore, no values needed to be removed or replaced.

3.2 Duplicate Records

Duplicate records are repeated rows containing the same information. They can affect calculations and produce incorrect results.

Duplicates were checked using:

```
print(df.duplicated().sum())
```

There were **no duplicate records** in the final dataset.

3.3 Incorrect Data Types

Data types describe the type of information stored in a column. For example, Views, Likes, and Comments should be numerical values, while Category should contain text.

The data types were checked using:

```
print(df.dtypes)
```

The dataset contained the correct data types, so no major conversion was required.

3.4 Inconsistent Values

Inconsistent values occur when the same category is written in different ways. For example, Tech, tech, and TECH could represent the same category.

The category values were checked and kept in a consistent format:

- Music
- Gaming
- Education
- Tech
- Comedy

This makes grouping and analysis easier.

4. Data Analysis Using NumPy & Pandas

Data analysis is the process of examining data to find useful information, patterns, and relationships. **NumPy** is useful for numerical calculations, while **Pandas** is useful for handling tables, filtering, grouping, and analyzing structured data.

4.1 Mean

The mean is the average value of a dataset. It is calculated by adding all the values and dividing the result by the number of values.

In this case study, the mean is used to calculate the average number of views, likes, and comments.

```
print("Average Views:", np.mean(views))
```

```
print("Average Likes:", np.mean(likes))
```

```
print("Average Comments:", np.mean(comments))
```

Results:

- **Average Views = 131,000**
- **Average Likes = 13,100**

- Average Comments = 1,310

4.2 Median

The median is the middle value when the data is arranged in ascending or descending order. It is useful because it represents the central value without being strongly affected by very high or low values.

```
print("Median Views:", np.median(views))
```

```
print("Median Likes:", np.median(likes))
```

```
print("Median Comments:", np.median(comments))
```

4.3 Standard Deviation

Standard deviation measures how spread out the values are from their average. A low standard deviation means that values are close to the mean, while a high standard deviation indicates greater variation.

```
print("Standard Deviation of Views:", np.std(views))
```

The views have noticeable variation because some videos have much higher views than others.

4.4 Grouping and Aggregation

Grouping allows us to divide the data into categories and calculate values for each group.

For example, the average views for each category can be calculated using:

```
df.groupby("Category")["Views"].mean()
```

Category Average Views

Comedy 145,000

Education 110,000

Gaming 130,000

Music 70,000

Tech 225,000

The result shows that **Tech has the highest average views**, while Music has the lowest average views.

4.5 Sorting and Filtering

Sorting arranges the data according to a particular column. For example, videos can be arranged from highest to lowest views.

```
df.sort_values("Views", ascending=False)
```

Filtering allows us to select only records that satisfy a particular condition.

For example:

```
df[df["Views"] > 150000]
```

This displays videos that have received more than 150,000 views.

4.6 Correlation

Correlation measures the relationship between two numerical variables. A positive correlation means that when one variable increases, the other tends to increase as well.

In this dataset, correlation is calculated between views, likes, and comments:

```
df[["Views", "Likes", "Comments"]].corr()
```

The analysis shows a strong positive relationship between these variables. This means videos with higher views generally also have higher likes and comments.

5. Data Visualization

Data visualization is the process of representing data using graphs and charts. It makes numerical information easier to understand and helps identify patterns and trends quickly.

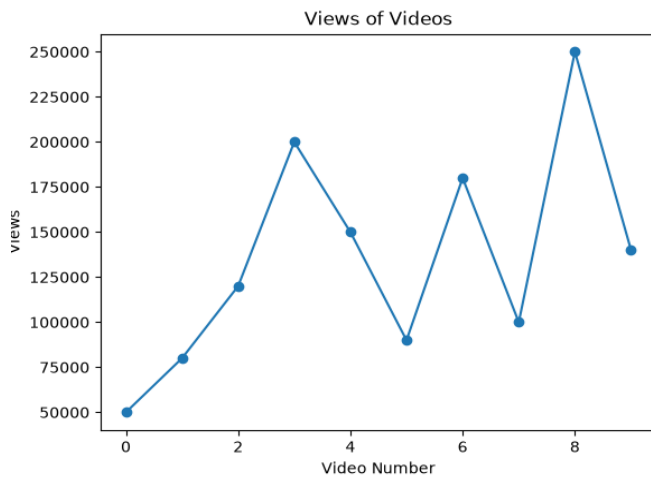
In this case study, **Matplotlib** and **Seaborn** are used for visualization.

5.1 Matplotlib

Line Chart

A line chart connects data points using lines. It is useful for observing changes and trends in data.

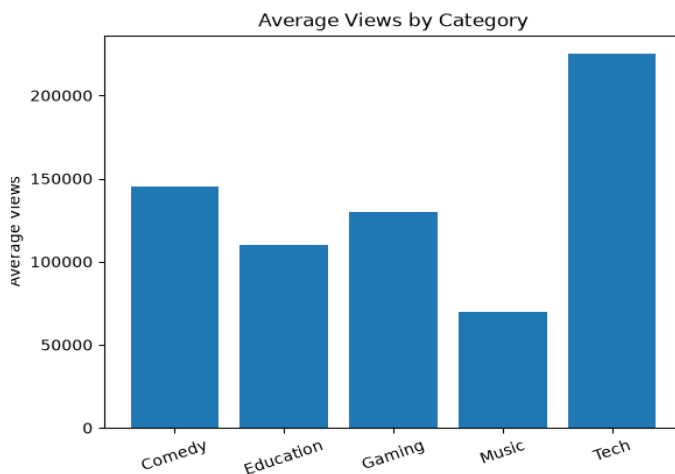
In this case study, the line chart represents the views of the 10 videos. It allows us to compare the view count from one video to another.



Bar Chart

A bar chart uses rectangular bars to compare different categories.

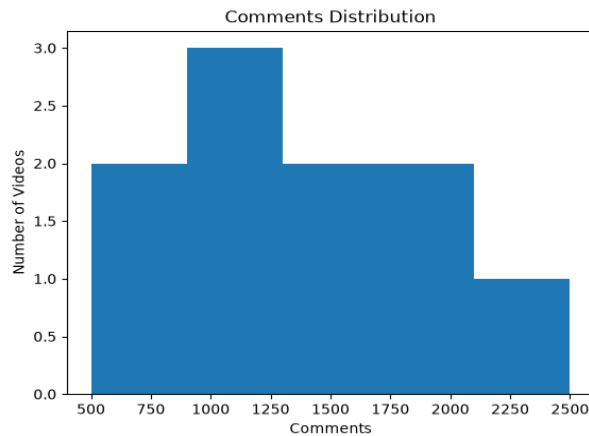
The bar chart in this case study compares the **average views of different YouTube categories**. It clearly shows that Tech has the highest average views.



Histogram

A histogram represents the frequency distribution of numerical data.

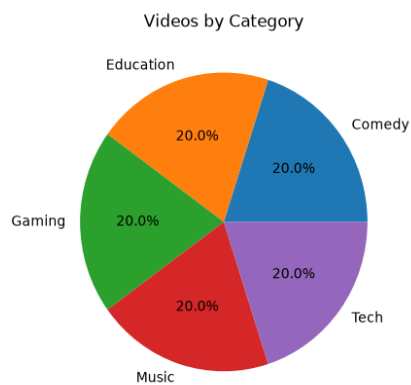
The histogram is used to show the distribution of comments among the videos. It helps us understand how frequently different comment ranges occur.



Pie Chart

A pie chart represents data as portions of a complete circle. It is useful for showing percentages or proportions.

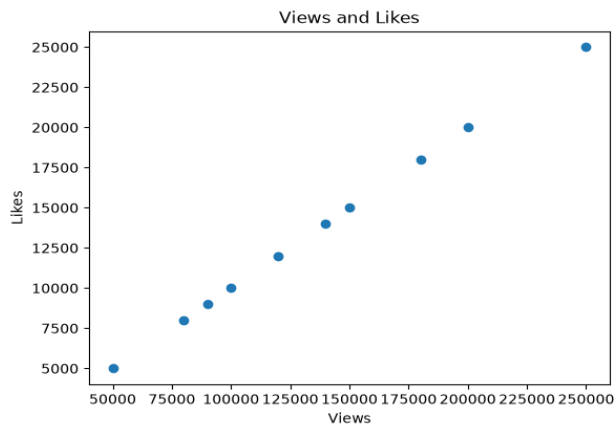
The pie chart represents the number of videos belonging to each category. Since every category has two videos, all categories have an equal share.



Scatter Plot

A scatter plot represents the relationship between two numerical variables using points.

In this case study, views are plotted against likes. The upward pattern of the points indicates that videos with more views generally receive more likes.

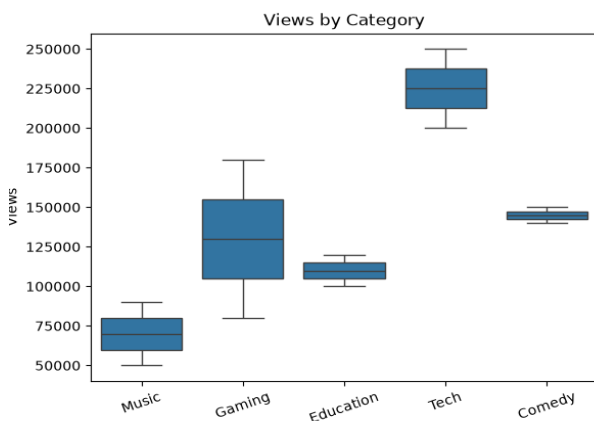


5.2 Seaborn

Box Plot

A box plot summarizes the distribution of numerical data. It can show the median, spread, and possible extreme values.

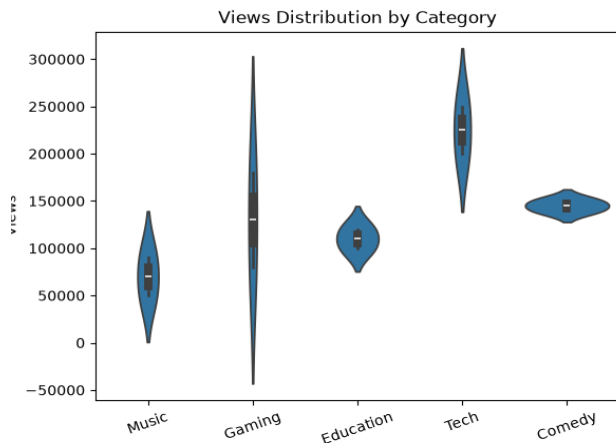
The box plot is used to compare the distribution of views among different YouTube categories.



Violin Plot

A violin plot combines information about the distribution and spread of data. It helps show where values are concentrated.

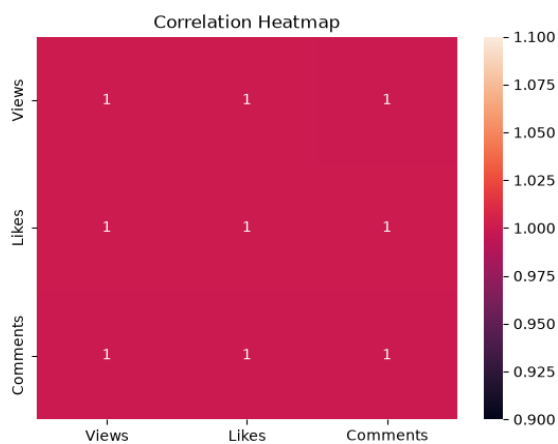
In this case study, the violin plot represents the distribution of views for each category.



Heatmap

A heatmap represents numerical values using different levels of shading. It is especially useful for displaying correlation between variables.

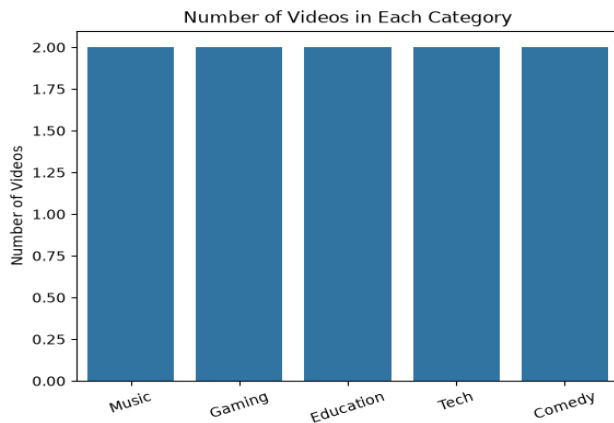
The heatmap in this case study shows the correlation between **Views, Likes, and Comments**.



Count Plot

A count plot displays the number of observations belonging to each category.

The count plot shows how many YouTube videos are present in each category. In our dataset, each category contains two videos.



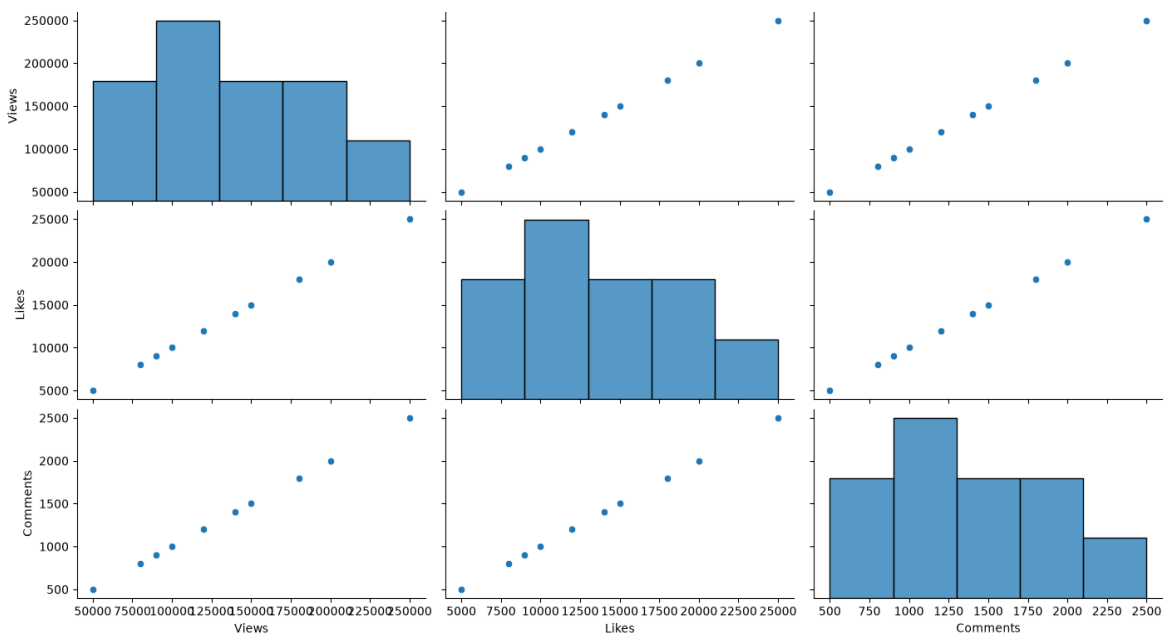
Pair Plot

A pair plot creates multiple plots to compare numerical variables with each other.

The pair plot is used to study the relationships between:

- Views
- Likes
- Comments

It provides a quick overall view of the relationships present in the dataset.



6. Findings

Based on the numerical analysis and visualizations, the following observations were made:

1. The dataset contains **10 YouTube videos** belonging to five different categories.
 2. The overall average number of views is **131,000**.
 3. **Tech videos have the highest average views**, with 225,000 average views.
 4. **Music videos have the lowest average views**, with 70,000 average views.
 5. Videos with higher views generally receive more likes.
 6. Views, likes, and comments show a **strong positive relationship** in the dataset.
 7. The scatter plot clearly shows that likes increase as the number of views increases.
 8. The count plot shows that each category contains an equal number of videos.
-

7. Conclusion

This case study analyzed YouTube video performance using **NumPy, Pandas, Matplotlib, and Seaborn**. The dataset was first checked for common data quality issues such as missing values, duplicate records, incorrect data types, and inconsistent category names.

Statistical analysis was then performed using mean, median, standard deviation, grouping, sorting, filtering, and correlation. The results showed that Tech videos had the highest average number of views in the dataset.

The visualizations made it easier to understand the differences between categories and the relationships between views, likes, and comments. Overall, the case study demonstrates how **Python-based data analysis and visualization can be used to extract useful information and patterns from YouTube data**.

8. Appendix – Python Code

This section contains the complete Python program used for data cleaning, analysis, and visualization of the YouTube dataset. The program uses **NumPy, Pandas, Matplotlib, and Seaborn**.

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# YouTube data
```

```
categories = np.array([  
    "Music", "Gaming", "Education", "Tech", "Comedy",  
    "Music", "Gaming", "Education", "Tech", "Comedy"  
])
```

```
views = np.array([  
    50000, 80000, 120000, 200000, 150000,  
    90000, 180000, 100000, 250000, 140000  
])
```

```
likes = np.array([  
    5000, 8000, 12000, 20000, 15000,  
    9000, 18000, 10000, 25000, 14000  
])
```

```
comments = np.array([  
    500, 800, 1200, 2000, 1500,  
    900, 1800, 1000, 2500, 1400  
])
```

```
])
```

```
# Creating DataFrame
```

```
df = pd.DataFrame({  
    "Category": categories,  
    "Views": views,  
    "Likes": likes,  
    "Comments": comments  
})
```

```
print("YouTube Data")
```

```
print(df)
```

```
# Data Cleaning
```

```
print("\nMissing Values")
```

```
print(df.isnull().sum())
```

```
print("\nDuplicate Records")
```

```
print(df.duplicated().sum())
```

```
print("\nData Types")
```

```
print(df.dtypes)
```

```
print("\nCategories")
```

```
print(df["Category"].unique())
```

```
# Data Analysis
```

```
print("\nAverage Views:", np.mean(views))
```

```
print("Average Likes:", np.mean(likes))
```

```
print("Average Comments:", np.mean(comments))
```

```
print("\nMedian Views:", np.median(views))
```

```
print("Median Likes:", np.median(likes))
```

```
print("Median Comments:", np.median(comments))
```

```
print("\nStandard Deviation of Views:", np.std(views))
```

```
print("\nAverage Views by Category")
```

```
average_views = []
```

```
for category in np.unique(categories):
```

```
    category_views = views[categories == category]
```

```
average = np.mean(category_views)
```

```
average_views.append(average)
```

```
print(category, ":", average)
```

Grouping using Pandas

```
print("\nGroup by Category")
```

```
print(df.groupby("Category")["Views"].mean())
```

Sorting

```
print("\nVideos Sorted by Views")
```

```
print(df.sort_values("Views", ascending=False))
```

Filtering

```
print("\nVideos with More Than 150000 Views")
```

```
print(df[df["Views"] > 150000])
```

Correlation

```
print("\nCorrelation")
```

```
print(df[["Views", "Likes", "Comments"]].corr())
```

1. Line Chart

```
plt.plot(views, marker="o")  
  
plt.title("Views of Videos")  
  
plt.xlabel("Video Number")  
  
plt.ylabel("Views")  
  
plt.show()
```

2. Bar Chart

```
plt.bar(np.unique(categories), average_views)  
  
plt.title("Average Views by Category")  
  
plt.xlabel("Category")  
  
plt.ylabel("Average Views")  
  
plt.xticks(rotation=20)  
  
plt.show()
```

3. Histogram

```
plt.hist(comments, bins=5)  
  
plt.title("Comments Distribution")  
  
plt.xlabel("Comments")
```

```
plt.ylabel("Number of Videos")
```

```
plt.show()
```

4. Pie Chart

```
category_names, category_count = np.unique(  
    categories, return_counts=True  
)
```

```
plt.pie(  
    category_count,  
    labels=category_names,  
    autopct="%1.1f%%"  
)
```

```
plt.title("Videos by Category")
```

```
plt.show()
```

5. Scatter Plot

```
plt.scatter(views, likes)
```

```
plt.title("Views and Likes")
```

```
plt.xlabel("Views")
```

```
plt.ylabel("Likes")
```

```
plt.show()
```

```
# 6. Box Plot
```

```
sns.boxplot(x=categories, y=views)
```

```
plt.title("Views by Category")
```

```
plt.xlabel("Category")
```

```
plt.ylabel("Views")
```

```
plt.xticks(rotation=20)
```

```
plt.show()
```

```
# 7. Violin Plot
```

```
sns.violinplot(x=categories, y=views)
```

```
plt.title("Views Distribution by Category")
```

```
plt.xlabel("Category")
```

```
plt.ylabel("Views")
```

```
plt.xticks(rotation=20)
```

```
plt.show()
```

```
# 8. Heatmap
```

```
data = np.array([views, likes, comments])
```

```
correlation = np.corrcoef(data)
```

```
sns.heatmap(  
    correlation,  
    annot=True,  
    xticklabels=["Views", "Likes", "Comments"],  
    yticklabels=["Views", "Likes", "Comments"]  
)  
  
plt.title("Correlation Heatmap")  
  
plt.show()
```

9. Count Plot

```
sns.countplot(x=categories)  
  
plt.title("Number of Videos in Each Category")  
  
plt.xlabel("Category")  
  
plt.ylabel("Number of Videos")  
  
plt.xticks(rotation=20)  
  
plt.show()
```

10. Pair Plot

```
data_for_pairplot = pd.DataFrame({
```

"Views": views,

"Likes": likes,

"Comments": comments

})

sns.pairplot(data_for_pairplot)

plt.show()